

TCS302: Data Structures with C

Semester-Wise Lecture Plan

Programme: B.Tech. Computer Science and Engineering

Semester: III

Credits: 3

Total Teaching Hours: 46

Teaching Load: Approximately 3 lectures per week

The plan follows the five-unit syllabus and prescribed teaching hours: Unit I-10 hours, Unit II-10 hours, Unit III-9 hours, Unit IV-9 hours, and Unit V-8 hours.

Unit I: Introduction, Arrays, Stacks and Recursion

Teaching Hours: 10

Course Outcome: CO1

Lecture	Topic to be Covered	Teaching-Learning Method
1	Introduction to data structures, basic terminology and elementary data organisation	Lecture and real-world examples
2	Data-structure operations and Abstract Data Type concept	Lecture and classroom discussion
3	Pointers, structures and dynamic memory allocation in C	Code demonstration
4	Algorithm complexity, time complexity, space complexity and time-space trade-off	Analytical examples
5	Array definition, representation and one-dimensional arrays	Lecture and C programming examples
6	Multidimensional arrays and address calculation	Numerical problem-solving
7	Array applications, arrays as parameters, ordered lists and sparse matrices	Demonstration and practice
8	Stack ADT, array representation, push, pop, peek, overflow and underflow	Algorithm and code demonstration
9	Linked representation of stacks; infix-to-postfix and infix-to-prefix conversion	Problem-solving session
10	Postfix evaluation, recursion in C, Tower of Hanoi and tail recursion	Coding and recursive-trace demonstration

Self-Learning Activity: Implement array and stack operations and analyse their time and space complexity.

Team Activity: Discuss the selection of appropriate data structures for real-world problems.

Unit II: Queues and Linked Lists

Teaching Hours: 10

Course Outcome: CO2

Lecture	Topic to be Covered	Teaching-Learning Method
11	Queue ADT, terminology and array representation	Lecture and examples
12	Queue operations: create, enqueue, dequeue, full and empty conditions	Algorithm and C implementation
13	Circular queue and its implementation	Code demonstration
14	Deque, priority queue and linked representation of queues	Comparative explanation
15	Introduction to linked lists, node structure and dynamic memory allocation	Lecture and coding
16	Singly linked-list creation, traversal and searching	Algorithm demonstration
17	Insertion operations in a singly linked list	Coding and dry run
18	Deletion operations, overflow and underflow in linked lists	Coding and problem-solving
19	Doubly linked lists and two-way header lists	Diagram-based explanation
20	Linked list in arrays, polynomial representation and addition, generalized linked lists	Application-based discussion

Self-Learning Activity: Implement queues and linked lists with their associated operations in C.

Team Activity: Develop a queue- or linked-list-based solution for a practical problem.

Unit III: Trees, Searching and Hashing

Teaching Hours: 9

Course Outcome: CO3

Lecture	Topic to be Covered	Teaching-Learning Method
21	Tree terminology, binary trees and applications	Lecture and tree diagrams
22	Complete and extended binary trees; array and linked representations	Diagram and representation exercises
23	Binary-tree traversal: preorder, inorder and postorder	Algorithm and recursive implementation
24	Algebraic-expression trees and priority queues	Application-based examples
25	Threaded binary trees and traversal of threaded trees	Diagram-based explanation
26	Huffman algorithm and construction of Huffman trees	Problem-solving activity
27	Sequential search and binary search; comparison and complexity analysis	Algorithm comparison
28	Hash tables, hash functions and hash-table implementation	Lecture and numerical examples
29	Collision-resolution strategies: chaining, linear probing, quadratic probing and double hashing	Problem-solving and coding

Self-Learning Activity: Implement binary trees, searching techniques and hash tables.

Team Activity: Develop a small application using Huffman coding or hashing.

Unit IV: Sorting and Binary Search Trees

Teaching Hours: 9

Course Outcome: CO4

Lecture	Topic to be Covered	Teaching-Learning Method
30	Introduction to sorting, sorting terminology, stability and efficiency	Lecture and comparison
31	Insertion sort: algorithm, implementation and complexity	Dry run and coding
32	Bubble sort: algorithm, implementation and complexity	Dry run and coding
33	Quick sort, partitioning and complexity analysis	Algorithm demonstration
34	Two-way merge sort and merge operation	Recursive demonstration
35	Heap sort and binary heaps	Heap visualisation and coding
36	Sorting on different keys and practical considerations for internal sorting	Case-based discussion
37	Binary Search Tree: properties, searching and insertion	Diagram and C implementation
38	Deletion in BST, path length, complexity analysis and AVL-tree rotations	Problem-solving and demonstration

Self-Learning Activity: Implement and compare sorting algorithms and BST operations.

Team Activity: Compare the running time of different sorting algorithms using sample datasets.

Unit V: File Structures and Graphs

Teaching Hours: 8

Course Outcome: CO5

Lecture	Topic to be Covered	Teaching-Learning Method
39	Physical storage media, file organisation and organisation of records into blocks	Lecture and examples
40	Sequential files, primary indices and secondary indices	Diagram-based explanation
41	B-tree and B+ tree index files	Tree construction exercises
42	File indexing, hashing and comparison of indexing and hashing	Comparative discussion
43	Graph terminology, graph representations and bipartite-graph checking	Diagram and problem-solving
44	Breadth-First Search and Depth-First Search	Algorithm and C implementation
45	Disjoint sets, union-find, path compression and union by rank/size	Code demonstration
46	Minimum Spanning Trees: Kruskal's and Prim's algorithms	Numerical problem-solving

Self-Learning Activity: Implement graph traversals, disjoint sets and minimum-spanning-tree algorithms.

Team Activity: Identify and solve a real-world problem using graphs.

Semester-Wise Weekly Distribution

Semester Week	Lectures	Unit and Coverage
Week 1	1-3	Unit I: Introduction, ADT, pointers and dynamic memory
Week 2	4-6	Unit I: Complexity and arrays
Week 3	7-9	Unit I: Array applications and stacks
Week 4	10-12	Unit I completion and Unit II introduction
Week 5	13-15	Unit II: Circular queue, deque, priority queue and linked lists
Week 6	16-18	Unit II: Linked-list traversal, insertion and deletion
Week 7	19-21	Unit II completion and Unit III introduction
Week 8	22-24	Unit III: Binary-tree representations and traversals
Week 9	25-27	Unit III: Threaded trees, Huffman coding and searching
Week 10	28-30	Unit III completion and Unit IV introduction
Week 11	31-33	Unit IV: Insertion, bubble and quick sort
Week 12	34-36	Unit IV: Merge sort, heap sort and sorting considerations
Week 13	37-39	Unit IV completion and Unit V introduction
Week 14	40-42	Unit V: File indexing, B-tree and B+ tree
Week 15	43-45	Unit V: Graphs, BFS, DFS and disjoint sets
Week 16	46 and revision	MST algorithms, course revision and doubt-clearing session

Suggested Assessment Distribution

- **After Unit I:** Quiz on arrays, stacks and recursion
- **After Unit II:** Programming assignment on queues and linked lists
- **After Unit III:** Class test on trees, searching and hashing
- **After Unit IV:** Sorting-performance comparison activity
- **After Unit V:** Graph-based programming assignment and semester revision